

Clase Práctica : Seguridad

Tomás F. Melli

November 2025

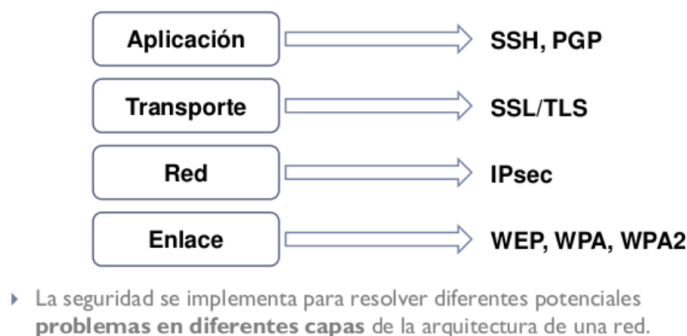
Índice

1	Introducción	2
1.1	Propiedades Fundamentales	2
1.2	Herramientas Fundamentales	2
1.3	Marco Normativo	3
2	Firma y Certificado Digital	3
2.1	Criptografía Simétrica	3
2.2	Criptografía Asimétrica	3
2.3	Notación Criptográfica Formal	4
2.4	Funciones de hash	5
2.5	Firma Digital	5
2.5.1	Proceso de generación de una firma	6
2.5.2	Verificación en el receptor	6
2.5.3	Autenticidad y No Repudio	6
2.5.4	El problema de la distribución de claves públicas	7
2.5.5	Soluciones: PKI y Web of Trust	7
2.6	Certificados Digitales y PKI	7
2.6.1	Verificación del Certificado	8
2.7	Certificados y PKI en la práctica	8
2.7.1	Cadena de Certificación (Chain of Trust)	8
2.7.2	Escenario Real con Múltiples CAs Raíz	9
2.7.3	Esquema de flujo básico dentro de una PKI	10
3	Ejercicio 1	10
4	Ejercicio 2	12
5	SSL/TLS Seguridad en la Capa de Transporte	12
5.1	Proceso de Handshake en SSL/TLS	13

1 Introducción

La Seguridad de la Información se basa en garantizar un conjunto de propiedades esenciales que protegen los datos y las comunicaciones frente a accesos indebidos, manipulaciones o disputas. En el contexto de la materia nos interesará ver la seguridad en las distintas capas:

Protocolos y Capas



- **Capa de Aplicación (SSH, PGP)**

En esta capa se protegen las aplicaciones directamente (por ejemplo, acceso remoto o correo electrónico). Los protocolos funcionan de manera extremo a extremo, sin depender de lo que haga la red intermedia. SSH protege sesiones remotas y PGP protege correos mediante cifrado y firma. Proveen confidencialidad, integridad, autenticación y no repudio.

- **Capa de Transporte (SSL/TLS)**

Aquí se protege la comunicación entre procesos, como la interacción entre un navegador y un servidor web. SSL/TLS cifra y autentica la conexión a nivel de transporte (por ejemplo, HTTPS). Proporciona confidencialidad del canal, integridad de datos y autenticación del servidor (y opcionalmente del cliente).

- **Capa de Red (IPsec)**

En esta capa se protegen los paquetes a nivel IP, sin importar qué aplicación o protocolo de transporte se esté usando. IPsec puede cifrar y autenticar tráfico IP completo, siendo fundamental en implementaciones de VPN. Garantiza integridad, autenticación y confidencialidad.

- **Capa de Enlace (WEP, WPA, WPA2)**

Se protege la comunicación a nivel del medio físico, especialmente en redes inalámbricas. Protocolos como WEP, WPA y WPA2 cifran tramas Wi-Fi. Proveen control de acceso y confidencialidad sobre comunicaciones inalámbricas.

1.1 Propiedades Fundamentales

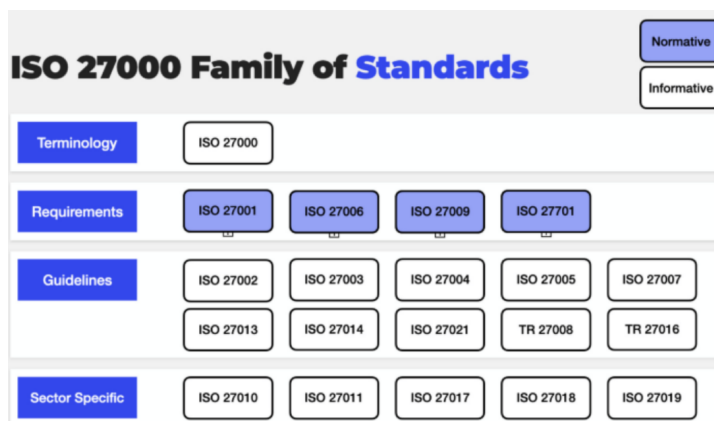
- **Confidencialidad:** asegura que la información sólo pueda ser interpretada por las partes autorizadas. Previene el acceso indebido mediante criptografía, controles de acceso o canales seguros.
- **Integridad:** garantiza que los mensajes no sean modificados, sustituidos o alterados durante su transmisión o almacenamiento. Permite verificar que el contenido recibido es exactamente el enviado.
- **Autenticación:** permite comprobar la identidad del emisor o receptor de un mensaje, asegurando que quien afirma ser el origen del mensaje realmente lo es.
- **No repudio:** evita que una parte niegue posteriormente haber participado en una transacción o haber enviado un mensaje. Requiere evidencia verificable, como una firma digital.

1.2 Herramientas Fundamentales

- **Criptografía:** disciplina que diseña protocolos para impedir el acceso no autorizado a comunicaciones privadas. Constituye la base matemática de los mecanismos de confidencialidad, autenticación, integridad y no repudio.
- **Esquema de Firma Digital:** versión electrónica de una firma manuscrita. Usa criptografía asimétrica y funciones hash para garantizar autenticidad, integridad y no repudio de documentos electrónicos.
- **Certificado Digital:** documento electrónico emitido por una Autoridad Certificante (CA) que vincula una identidad con una clave pública, permitiendo confiar en la autenticidad de dicha clave.

1.3 Marco Normativo

La familia de normas **ISO/IEC 27000** proporciona el marco internacional para la gestión de la Seguridad de la Información y la Ciberseguridad. Define conceptos, requisitos y controles para implementar sistemas de gestión de seguridad (SGSI).

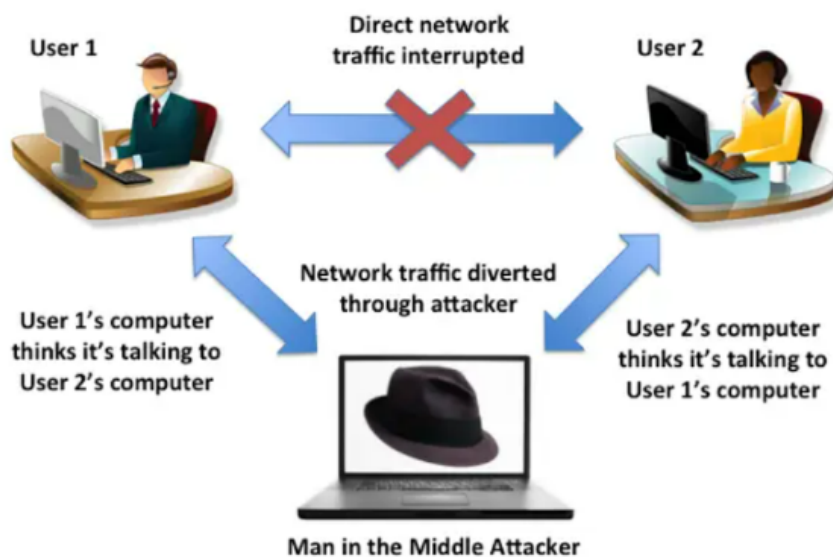


2 Firma y Certificado Digital

2.1 Criptografía Simétrica

La criptografía simétrica utiliza una única clave secreta compartida por ambas partes para cifrar y descifrar mensajes. Su principal ventaja es la eficiencia computacional, pero presenta limitaciones importantes en términos de seguridad.

¿Qué pasa si...?



- **Integridad:** es posible modificar los mensajes durante la transmisión y no existe un mecanismo nativo dentro de la criptografía simétrica para detectar dicha modificación. No se puede comprobar integridad.
- **Autenticación:** al ser una clave compartida, ninguna de las partes puede demostrar que fue la otra quien generó un mensaje. No es posible autenticar identidades con esta técnica por sí sola.

Estas limitaciones hacen que la criptografía simétrica, aunque útil para confidencialidad, requiera mecanismos adicionales (como MACs o HMACs) para lograr integridad y autenticación.

2.2 Criptografía Asimétrica

La criptografía asimétrica, también llamada criptografía de clave pública, utiliza dos claves diferentes:

- una **clave pública** K_A^+ , que se comparte libremente;
- una **clave privada** K_A^- , que se mantiene en secreto.

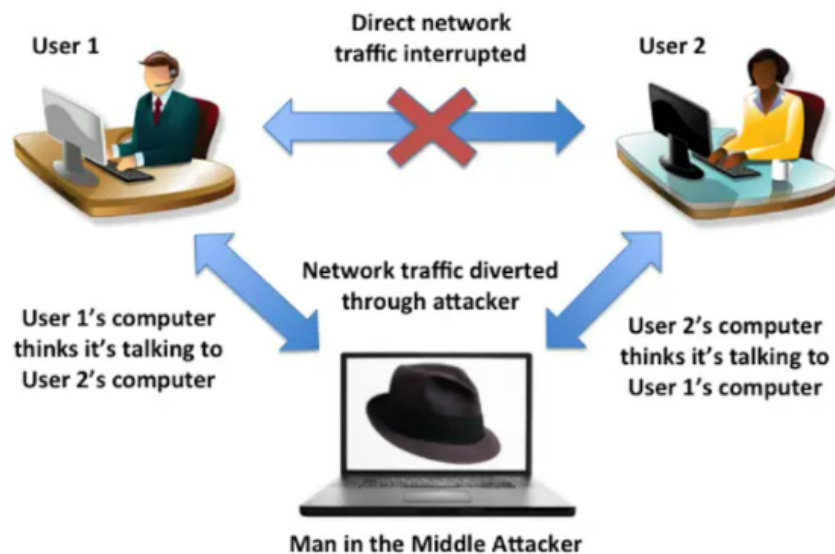
Propiedades

- Permite garantizar **confidencialidad** cuando un mensaje se cifra con la clave pública del receptor.
- Permite garantizar **autenticación** cuando un mensaje (o su hash) se cifra con la clave privada del emisor.
- Dependiendo del modo en el que se utilicen las claves, se pueden lograr distintos objetivos.

Desventajas

- **Relación clave–identidad vulnerable:** sin un mecanismo externo de validación, no sabemos si una clave pública realmente pertenece a quien dice ser su dueño. Esto habilita ataques de intermediario (MITM).
- **Costo computacional elevado:** los algoritmos asimétricos (como RSA) son mucho más lentos que los simétricos.

¿Qué pasa si...?



A pesar de sus ventajas, la criptografía asimétrica por sí sola tampoco provee integridad del mensaje completo. Si el mensaje se envía sin técnicas adicionales (como firmas digitales), es posible modificarlo sin detección. La integridad requiere funciones hash y/o firmas digitales.

2.3 Notación Criptográfica Formal

Un sistema criptográfico puede describirse formalmente mediante la tupla:

$$(M, K, C, E, D)$$

donde:

- M : conjunto de mensajes válidos en texto plano.
- K : conjunto de claves.
- C : conjunto de textos cifrados (criptogramas).
- $E : M \times K \rightarrow C$: función de encriptación.
- $D : C \times K \rightarrow M$: función de descryptación.

Cómo utilizar la notación?

- Mensaje: M
- Criptograma: C
- Digesto (hash): D

- Clave simétrica: K
- Clave pública de A : K_A^+
- Clave privada de A : K_A^-

Operaciones

- **Encriptar** un mensaje M usando la clave Q :

$$E_Q(M) = C, \quad Q \in \{K, K_A^+, K_A^-\}$$

- **Desencriptar** un criptograma C usando la clave Q :

$$D_Q(C) = M, \quad Q \in \{K, K_A^+, K_A^-\}$$

- **Aplicar una función de hash criptográfico:**

$$H(M) = D$$

Aclaración importante

La notación combina de forma abstracta algoritmos y claves para mostrar posibles operaciones. Sin embargo, en la práctica:

- No todas las claves pueden usarse con todos los algoritmos.
- Debe aclararse explícitamente qué algoritmo se utiliza en cada caso (AES, RSA, SHA-3, etc.).

2.4 Funciones de hash

Las funciones de hash son una **primitiva criptográfica** que garantiza la **integridad** de los datos.

- Una función hash se define como:

$$h : A \rightarrow B$$

donde A es el conjunto de entradas de *tamaño arbitrario* y B es el conjunto de salidas de *longitud fija*.

- Dado $x \in A$, el valor $h(x)$ es **fácil de computar**.
- Es **no inversible**: para todo $y \in B$ resulta computacionalmente inviable encontrar un $x \in A$ tal que:

$$h(x) = y$$

- Es **resistente a colisiones**: es computacionalmente inviable encontrar $x, \hat{x} \in A$ tales que:

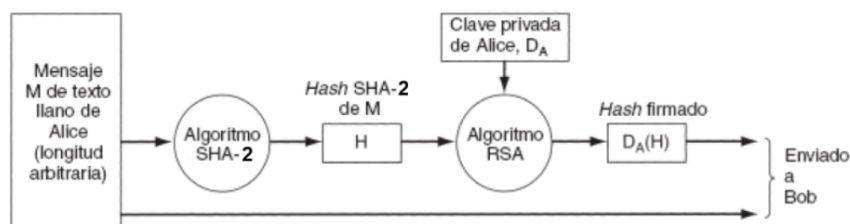
$$x \neq \hat{x} \quad \text{y} \quad h(x) = h(\hat{x})$$

- Ejemplos de funciones de hash:

HAVAL, MD5, SHA-1, SHA-2, SHA-3, BLAKE2

- **SHA-2 (incluyendo SHA-256)** es hoy el mínimo recomendable.

2.5 Firma Digital



La *firma digital* es un mecanismo criptográfico que permite garantizar la integridad, autenticidad y no repudio de un mensaje. Su funcionamiento se basa en el uso combinado de funciones hash y criptografía asimétrica.

2.5.1 Proceso de generación de una firma

El emisor desea enviar un mensaje M . Para firmarlo realiza el siguiente procedimiento:

1. Calcula el hash del mensaje:

$$h = H(M)$$

2. Encripta el hash utilizando su clave privada K_E^- :

$$\text{Firma} = E_{K_E^-}(h)$$

3. Envía al receptor el par:

$$(M, \text{Firma})$$

2.5.2 Verificación en el receptor

El receptor, al recibir (M, Firma) , puede comprobar la integridad del mensaje mediante:

1. Calcular localmente:

$$h' = H(M)$$

2. Desencriptar la firma usando la clave pública del emisor K_E^+ :

$$h'' = D_{K_E^+}(\text{Firma})$$

3. Comparar:

$$h' = h'' \quad ?$$

- Si coinciden, el mensaje no fue modificado (integridad).
- Además, la firma pudo ser generada únicamente por quien posee la clave privada del emisor (autenticidad y no repudio).

¿Por qué no se firma directamente el mensaje completo?

No se encripta el mensaje completo por dos razones principales:

- **Costo computacional:** cifrar mensajes largos con criptografía asimétrica es extremadamente ineficiente.
- **Propósito:** la firma busca autenticar *el contenido*, no ocultarlo. Para confidencialidad se emplean otros mecanismos (p.ej. criptografía simétrica).

Por eso, solo se firma el hash del mensaje, que es corto, fijo y representa unívocamente su contenido.

¿Qué pasa si alguien modifica el mensaje?

Aunque un atacante pueda interceptar y modificar el mensaje M , no podrá generar una nueva firma válida porque no posee la clave privada del emisor. El receptor detectará inmediatamente la alteración cuando:

$$H(M_{\text{alterado}}) \neq D_{K_E^+}(\text{Firma})$$

Con esto, la integridad queda garantizada.

2.5.3 Autenticidad y No Repudio

La firma digital también proporciona:

- **Autenticidad:** solo el dueño de la clave privada pudo haber generado la firma.
- **No repudio:** el emisor no puede negar que envió el mensaje, ya que la firma lo vincula de manera única con su clave privada.

2.5.4 El problema de la distribución de claves públicas

Hasta ahora asumimos que el receptor ya conoce la clave pública del emisor. Sin embargo, esto plantea una pregunta crítica:

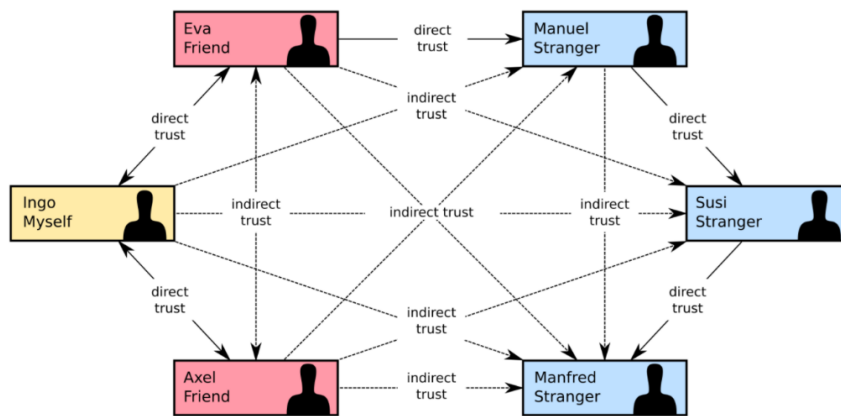
¿Cómo obtiene cada entidad la clave pública de la otra de forma confiable?

Si un atacante suplanta la clave pública, puede realizar un ataque de intermediario (MITM) y romper la autenticidad del sistema.

2.5.5 Soluciones: PKI y Web of Trust

Existen dos enfoques principales:

- **PKI (Public Key Infrastructure):** un sistema centralizado con Autoridades de Certificación que validan identidades y emiten certificados digitales.
- **Web of Trust (PGP):** sistema descentralizado donde los usuarios firman sus claves entre sí; frecuentemente se organizan *fiestas de firmado* para construir redes de confianza.



Ambos mecanismos buscan resolver el mismo problema: asociar de manera confiable identidades con claves públicas.

2.6 Certificados Digitales y PKI

Los *certificados digitales* son componentes fundamentales para establecer comunicaciones seguras en redes modernas. Su función principal es vincular de manera confiable la identidad de una entidad con su correspondiente clave pública. Este mecanismo se implementa mediante una *Infraestructura de Clave Pública* (PKI, Public Key Infrastructure), cuyo objetivo es gestionar claves, identidades y certificados en entornos distribuidos.

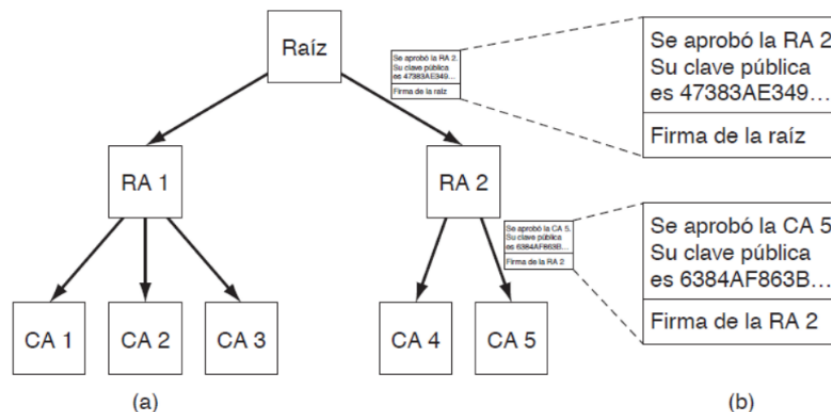
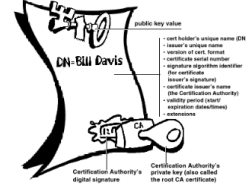


Figura 1-16. (a) Una PKI jerárquica. (b) Una cadena de certificados.

El estándar predominante para la definición de certificados digitales es **X.509v3**, especificado en la **RFC 6818**. Este estándar establece la estructura interna del certificado, sus campos, extensiones y el modo en que debe firmarse para garantizar su autenticidad.



Un certificado emitido para un cliente C_i puede representarse como:

$$(C_i; K_{C_i}^+; F_{CA,C_i})$$

donde:

- C_i : Identidad del cliente.
- $K_{C_i}^+$: Clave pública del cliente.
- F_{CA,C_i} : Firma digital realizada por la Autoridad Certificadora (CA).

La firma digital incluida en el certificado tiene la forma:

$$F_{CA,C_i} = E_{K_{CA}^-} (H(C_i + K_{C_i}^+))$$

Es decir, la Autoridad Certificadora aplica una función hash a la combinación de la identidad y la clave pública del cliente, y luego cifra ese hash utilizando su clave privada K_{CA}^- . Esto genera una firma verificable mediante la clave pública de la CA.

2.6.1 Verificación del Certificado

Para comprobar la validez del certificado de un cliente C_i , una entidad verificadora realiza los siguientes pasos:

1. Extrae del certificado los valores C_i , $K_{C_i}^+$ y la firma F_{CA,C_i} .
2. Calcula localmente el hash $H(C_i + K_{C_i}^+)$.
3. Descripta la firma utilizando la clave pública de la CA:

$$D_{K_{CA}^+} (F_{CA,C_i})$$

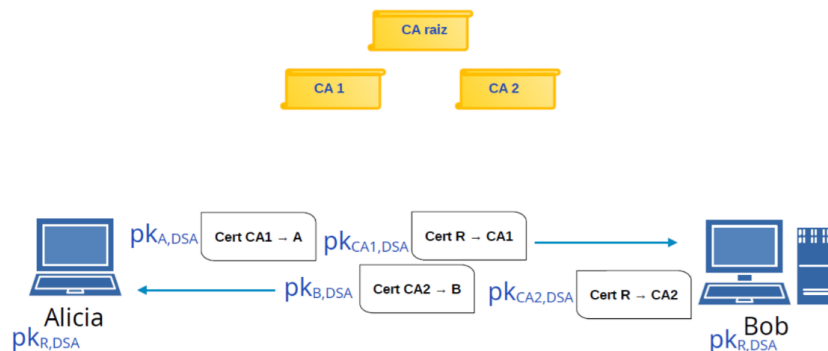
4. Compara ambos resultados:
 - Si coinciden, el certificado es auténtico y no ha sido modificado.
 - Si no coinciden, el certificado es inválido o fue alterado.

Este proceso garantiza:

- **Integridad** del contenido del certificado.
- **Autenticidad** de la clave pública del cliente.
- **Confianza** en la identidad del poseedor de la clave privada asociada.

2.7 Certificados y PKI en la práctica

2.7.1 Cadena de Certificación (Chain of Trust)



Esta imagen muestra cómo Alice y Bob verifican sus claves públicas utilizando una cadena de confianza formada por una CA raíz y CAs intermedias.

La estructura es:

- Una **CA raíz**, cuya clave pública está preinstalada en los sistemas.
- Dos **CAs intermedias**: CA1 y CA2.
- Usuarios: Alicia y Bob, cada uno con claves públicas firmadas por distintas CAs intermedias.

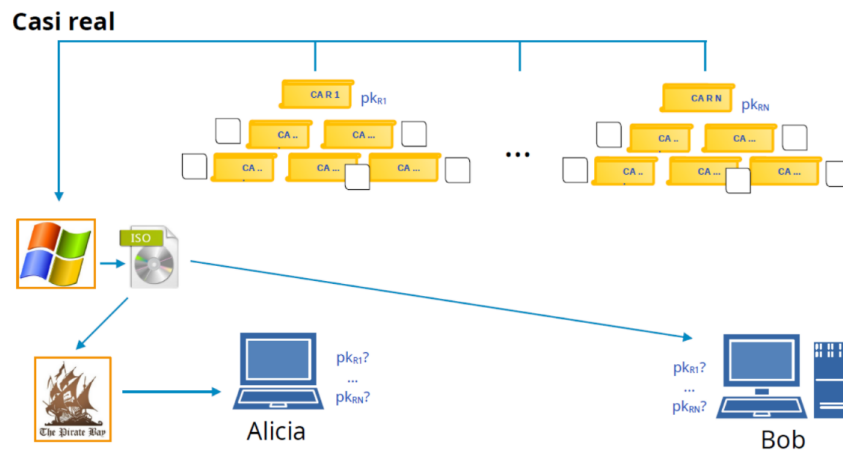
El proceso cuando Alicia quiere verificar la clave pública de Bob es:

1. Bob envía su certificado digital firmado por CA2.
2. Bob también envía el certificado de CA2 firmado por la CA raíz.
3. Alicia verifica:
 - Que la firma de CA2 fue realizada usando la clave pública de la CA raíz (que ya posee).
 - Que CA2 firmó correctamente el certificado de Bob.
4. Alicia concluye que la clave pública de Bob es auténtica.

El proceso para que Bob verifique a Alicia es análogo usando CA1.

La idea central es que la CA raíz funciona como autoridad máxima de confianza, y los certificados encadenados permiten verificar identidades sin que los usuarios deban conocerse previamente.

2.7.2 Escenario Real con Múltiples CAs Raíz



En la práctica, existen muchas CAs raíz distribuidas globalmente. Los sistemas operativos y navegadores almacenan un conjunto de claves públicas raíz confiables.

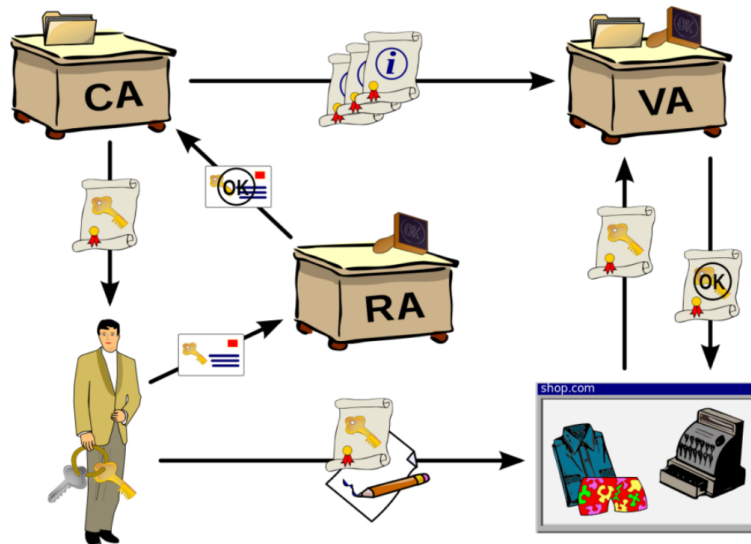
Cada CA raíz mantiene su propia jerarquía de CAs intermedias. Esto crea múltiples cadenas de certificación posibles.

El flujo conceptual de la imagen es:

- Existen múltiples CAs raíz ($CA_{R1}, CA_{R2}, \dots, CA_{RN}$).
- Cada una posee CAs intermedias que firman certificados para usuarios, software, sitios web, etc.
- Computadoras como las de Alicia o Bob confían automáticamente en todas las CAs raíz instaladas en su sistema.
- Cuando Alicia descarga un software (por ejemplo, una imagen ISO), puede verificar su firma digital porque la firma fue emitida por alguna CA que se encuentra en su almacén de confianza.

La idea central es que los sistemas modernos confían simultáneamente en muchas CAs, permitiendo validar firmas digitales emitidas por cualquiera de ellas.

2.7.3 Esquema de flujo básico dentro de una PKI



Los actores involucrados son:

- **CA (Certification Authority):** Firma certificados digitales.
- **RA (Registration Authority):** Verifica la identidad del usuario antes de solicitar la firma del certificado a la CA.
- **VA (Validation Authority):** Permite validar el estado de los certificados (OCSP/CRL).
- **Usuario:** Posee un par de claves y solicita un certificado.

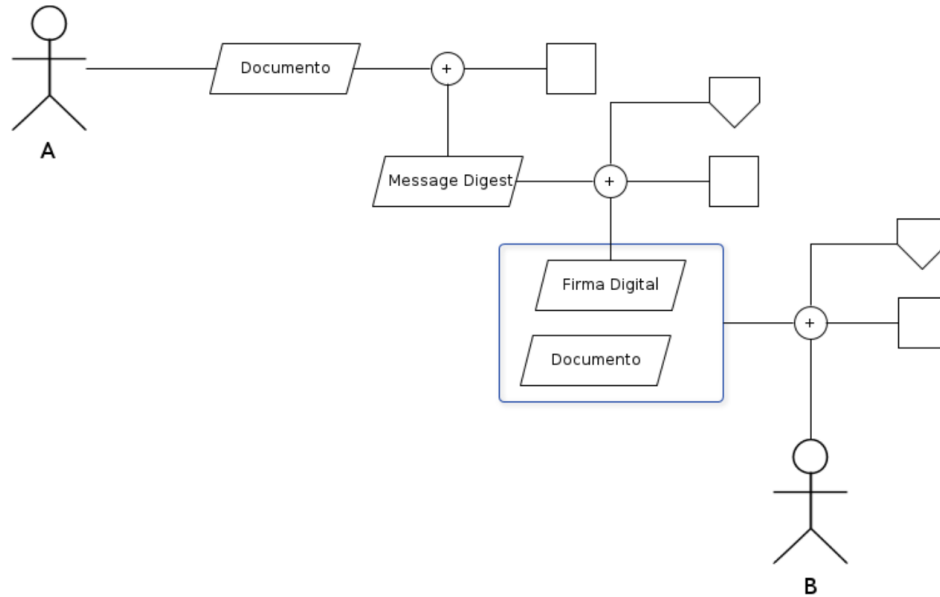
El proceso funciona de la siguiente forma:

1. El usuario genera un par de claves: privada (que guarda) y pública (que envía a la RA).
2. La RA verifica la identidad del usuario.
3. La RA envía la solicitud aprobada a la CA.
4. La CA firma la clave pública del usuario, generando un certificado digital.
5. El usuario recibe el certificado firmado.
6. Cuando el usuario se conecta a un sitio (por ejemplo, una tienda web), presenta su certificado.
7. El sitio valida el certificado utilizando la VA para revisar su vigencia o revocación.

La idea central es que la *confianza* se establece gracias a que la CA firma digitalmente la clave pública del usuario, permitiendo autenticidad e integridad dentro del sistema.

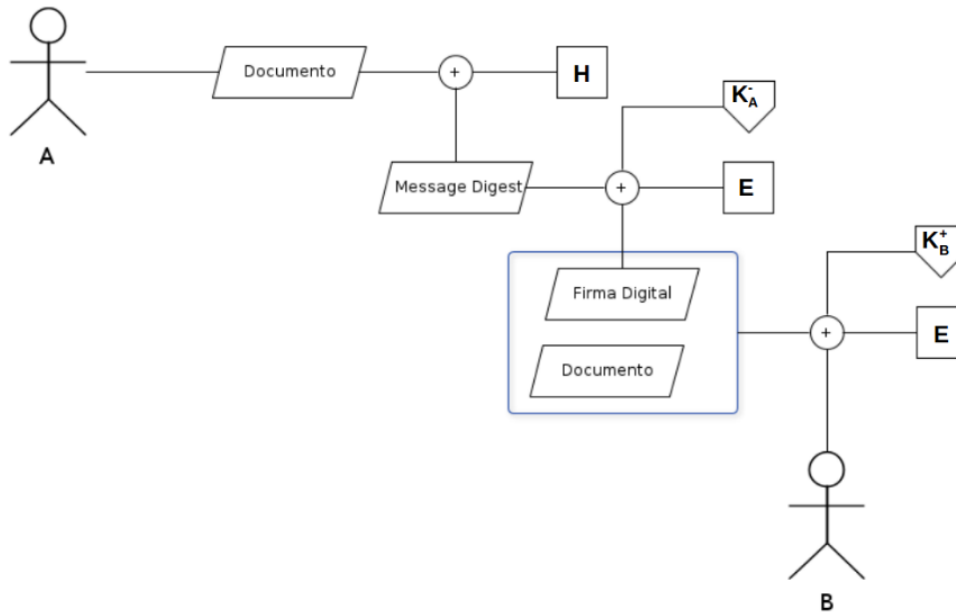
3 Ejercicio 1

A desea enviarle un mensaje a B de carácter importante. A quiere asegurarse de que nadie excepto B pueda leerlo y que B pueda confiar en que A fue quién envió el mensaje.



Basandose en el esquema a continuación, completando los cuadrados con algoritmos y las casitas con los parámetros adicionales que estos toman, explique cómo puede hacer A para construir el mensaje usando criptografía asimétrica, de manera de garantizar las propiedades de Confidencialidad y No repudio.

Sean K^A , la clave privada de A; K_B^+ , la clave pública de B; E un algoritmo de encriptación de clave asimétrica; y H una función de Hash, para garantizar Confidencialidad y Autenticación hay que hacer:

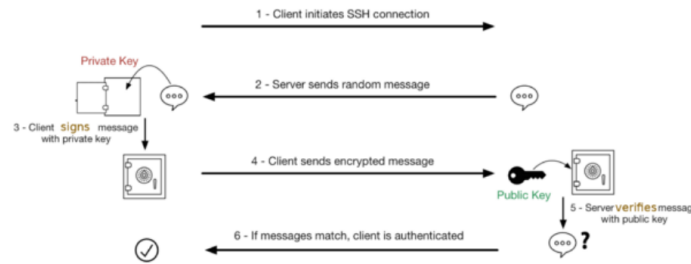


Explique qué debe hacer B para verificar que A fue quién envió el mensaje

Dado el criptograma C que recibe B, se debe descriptar usando un algoritmo de clave asimétrica D con su clave privada K_B^- , obteniendo $D_{K_B^-}(C) = M + F$, dónde M es el documento original que envía A, y F, la firma digital. Como B es el único capaz de descriptar el mensaje cifrado C, entonces queda garantizada la Confidencialidad. Luego, para verificar la firma, B debe descriptar la firma usando la clave pública de A (K_A^+) y comparar aplicando la misma función de Hash al documento: $D_{K_A^+}(F) == H(M)$, y así queda garantizado la autenticación de A sobre el mensaje enviado.

4 Ejercicio 2

Suponga que una compañía necesita implementar un sistema que garantice la autenticidad de sus clientes usando un sólo servicio instalado en un servidor.



Se sabe que ssh admite autenticación por clave pública y privada mediante el método de challenge-response. Explique dónde y cuántas claves públicas y privadas deberían instalarse para que se pueda garantizar la autenticidad de los clientes usando ssh.

En el caso de SSH, las claves deben instalarse previamente en cada cliente y servidor, dependiendo de quién debe garantizar la autenticidad y sobre quién. Para que se pueda garantizar la autenticidad de N clientes (C_i , $1 \leq i \leq N$), cada uno de ellos debe generar un par de claves pública y privada ($K_{C_i}^+$, $K_{C_i}^-$). Luego, deben instalarse las N claves públicas $K_{C_i}^+$ en el servidor, asociándolas a cada uno de los clientes.

Suponga ahora que los clientes necesitan garantizar la autenticidad del servidor de la compañía, ¿Dónde deberían instalarse las claves públicas y privadas?

Para garantizar la autenticidad del servidor, éste debe generar un par de claves pública y privada (K_S^+ , K_S^-) y, luego, debe instalarse K_S^+ en cada cliente.

También se puede garantizar autenticidad estableciendo una conexión SSL/TLS que usa un handshake seguro intercambiando certificados digitales. Explique cómo cambian la soluciones de los incisos a. y b. si la compañía dispone de un certificado digital propio firmado por una autoridad certificante.

Primero que nada, todas las máquinas (clientes y servidor) tienen que tener instalado un certificado de una autoridad certificante (CA) que contiene su clave pública K_{CA}^+ .

En el handshake SSL/TLS, la autenticación del servidor es obligatoria. Éste debe generar un par de claves pública y privada (K_S^+ , K_S^-), generar su certificado, hacerlo firmar por la CA e instalarlo en el servidor para su posterior envío en el Handshake SSL.

Luego, para garantizar la autenticidad de los clientes C_i , cada uno de estos debe generar un par de claves pública y privada ($K_{C_i}^+$, $K_{C_i}^-$), construir un certificado con la clave pública $K_{C_i}^+$ y hacerlo firmar por la autoridad certificante de manera que quede asociado con el nombre del cliente (C_i). El certificado que se debe instalar en cada cliente, por ejemplo, contendría la siguiente información:

$$(C_i, K_{C_i}^+, F_{CA, C_i})$$

donde

$$F_{CA, C_i} = E(H(C_i + K_{C_i}^+), K_{CA}^-)$$

donde E es un algoritmo de cifrado asimétrico y H es una función hash.

Luego, se instala cada certificado en cada cliente para que se envíe durante el Handshake SSL y para que el servidor pueda garantizar la autenticidad usando K_{CA}^+ para validar el certificado y, luego, $K_{C_i}^+$ para autenticar al cliente.

Si no se necesita autenticar clientes, se omite esta parte. En el handshake SSL/TLS, la autenticación del cliente es opcional.

5 SSL/TLS Seguridad en la Capa de Transporte

La seguridad en sistemas distribuidos puede implementarse en distintas capas del modelo de comunicaciones. No obstante, es importante destacar que la máxima seguridad se logra mediante mecanismos *end-to-end*, es decir, directamente entre las

aplicaciones en ambos extremos de la comunicación. En clases anteriores se han visto ejemplos de mecanismos que operan exclusivamente en la capa de aplicación.

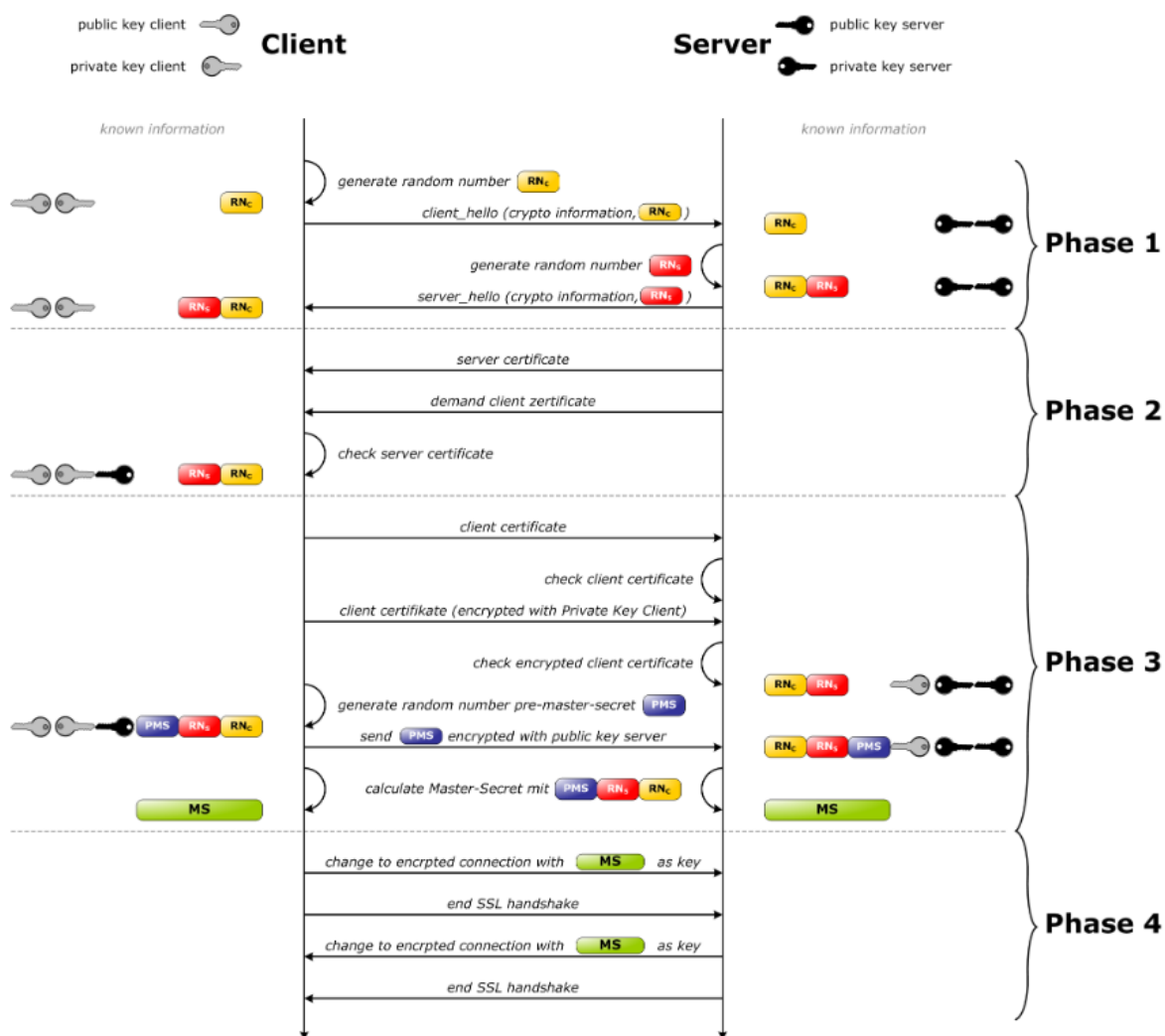
Sin embargo, también es posible incorporar funcionalidades de seguridad en capas inferiores. Un caso especialmente importante es la capa de transporte, donde surgen los protocolos SSL y TLS (Seguridad de la Capa de Transporte). Estos protocolos permiten asegurar la comunicación entre un cliente y un servidor de forma transparente para las aplicaciones que se ejecutan por encima.

SSL fue el primer protocolo ampliamente implementado con este propósito. Está compuesto por dos subprotocolos principales: uno destinado al establecimiento de una conexión segura (*handshake*) y otro encargado de la transmisión de datos una vez establecida dicha conexión.

Posteriormente surgió TLS como sucesor de SSL. La versión moderna, TLS 1.3, incorpora numerosas mejoras en eficiencia, seguridad criptográfica y reducción del número de mensajes de negociación. Sin embargo, estas mejoras también implican que TLS no sea interoperable con versiones antiguas de SSL.

Debido a esta incompatibilidad, la mayoría de los navegadores y clientes mantienen soporte para ambos protocolos. Durante la fase de negociación, TLS puede recurrir a SSL si el servidor no soporta versiones modernas. Este mecanismo de compatibilidad recibe el nombre de **SSL/TLS**.

5.1 Proceso de Handshake en SSL/TLS



El *handshake* es el procedimiento mediante el cual se establece una conexión segura entre cliente y servidor. El proceso consiste en una serie de pasos que permiten:

- acordar algoritmos criptográficos,
- autenticar a las partes,
- generar material criptográfico para la sesión.

Los pasos fundamentales del handshake son:

1. **Negociación de algoritmos:** En esta fase inicial el cliente y el servidor intercambian mensajes para acordar los algoritmos criptográficos que se utilizarán durante la conexión. Esto incluye métodos de cifrado simétrico, funciones de hash y mecanismos de intercambio de claves.
2. **Autenticación:** El servidor (y opcionalmente el cliente) se autentican mediante certificados digitales. Estos certificados se utilizan para verificar la identidad de cada extremo y garantizar la ausencia de ataques de intermediario (*man-in-the-middle*).
3. **Generación de la pre-master key:** El cliente genera una pre-master key y la envía al servidor cifrada con la clave pública del servidor. Esta operación utiliza criptografía asimétrica, lo que garantiza la confidencialidad del valor enviado.
4. **Cálculo de la clave maestra:** Tanto el cliente como el servidor calculan de manera independiente una clave maestra (master key) a partir de la pre-master key y los nonces intercambiados durante la etapa inicial de negociación. Este proceso asegura que ambos extremos obtengan exactamente el mismo valor sin necesidad de transmitirlo directamente.
5. **Cifrado simétrico de la sesión:** Una vez completado el handshake, la comunicación se cifra utilizando claves simétricas derivadas de la clave maestra. Esto se hace por razones de eficiencia: el cifrado simétrico es significativamente más rápido que el cifrado asimétrico, especialmente en conexiones de larga duración o con alto volumen de datos.